

## CM3031 Series CMSIS BSP Guide

Directory Introduction for 32-bit NuMicro™ Family

### Directory Information

Please extract the “CM3031\_Series\_BSP\_CMSIS\_V1.00.000.zip” file firstly, and then put the “CM3031\_Series\_BSP\_CMSIS\_V1.00.000” folder into the working folder (e.g. .\Nuvoton\BSP Library).

To experience the powerful features of CM3031 in few minutes, please select the sample code of your interest to download and execute on the CM3031 board. For example, you can try the TrustZone® technology in CM3031 that provides system-wide hardware isolation for trusted software with the sample code in the folder BSP\SampleCode\TrustZone. This folder includes the sample code of TrustZone® for collaborative secure software development, Hard Fault handling and a TrustZone® template. You can open the project files to build them with Keil® MDK, IAR or VSCode, and then download and trace them on the CM3031 board to see how the TrustZone® works.

This BSP folder contents:

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| <b>Document</b>   | Device driver reference manual and reversion history. |
| <b>Library</b>    | Device driver header and source files.                |
| <b>SampleCode</b> | Device driver sample code.                            |
| <b>ThirdParty</b> | Libraries of third parties.                           |

*The information described in this document is the exclusive intellectual property of Nuvoton Technology Corporation and shall not be reproduced without permission from Nuvoton.*

*Nuvoton is providing this document only for reference purposes of NuMicro microcontroller based system design. Nuvoton assumes no responsibility for errors or omissions.*

*All data and specifications are subject to change without notice.*

For additional information or questions, please contact: Nuvoton Technology Corporation.

[www.nuvoton.com](http://www.nuvoton.com)

## TABLE OF CONTENTS

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>DOCUMENT .....</b>                                                       | <b>4</b>  |
| <b>2</b> | <b>LIBRARY .....</b>                                                        | <b>5</b>  |
| <b>3</b> | <b>SAMPLE CODE.....</b>                                                     | <b>6</b>  |
| <b>4</b> | <b>THIRDPARTY.....</b>                                                      | <b>7</b>  |
| <b>5</b> | <b>SAMPLECODE\ISP.....</b>                                                  | <b>8</b>  |
| <b>6</b> | <b>SAMPLECODE\NUMAKER-M3334KIV2 .....</b>                                   | <b>9</b>  |
| <b>7</b> | <b>SAMPLECODE\POWERMANAGEMENT .....</b>                                     | <b>10</b> |
| <b>8</b> | <b>SAMPLECODE\STDDRIVER.....</b>                                            | <b>11</b> |
|          | System Manager (SYS) .....                                                  | 11        |
|          | Clock Controller (CLK) .....                                                | 11        |
|          | Flash Memory Controller (FMC).....                                          | 11        |
|          | General Purpose I/O (GPIO).....                                             | 12        |
|          | PDMA Controller (PDMA) .....                                                | 12        |
|          | Timer Controller (TIMER) .....                                              | 13        |
|          | Watchdog Timer (WDT) .....                                                  | 14        |
|          | Window Watchdog Timer (WWDT) .....                                          | 14        |
|          | Real Timer Clock (RTC) .....                                                | 14        |
|          | Basic PWM Generator and Capture Timer (BPWM).....                           | 14        |
|          | Enhance PWM Generator and Capture Timer (EPWM) .....                        | 14        |
|          | Enhanced Quadrature Encoder Interface (EQEI) .....                          | 15        |
|          | UART Interface Controller (UART).....                                       | 15        |
|          | I <sup>2</sup> S Controller (I <sup>2</sup> S).....                         | 16        |
|          | I <sup>3</sup> S Controller (I <sup>3</sup> S).....                         | 16        |
|          | Serial Peripheral Interface (SPI) .....                                     | 16        |
|          | Quad Serial Peripheral Interface (QSPI) .....                               | 17        |
|          | I <sup>2</sup> C Serial Interface Controller (I <sup>2</sup> C).....        | 18        |
|          | Universal Serial Control Interface Controller - UART Mode (USCI-UART) ..... | 18        |
|          | Universal Serial Control Interface Controller - SPI Mode (USCI-SPI).....    | 18        |

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| Universal Serial Control Interface Controller - I <sup>2</sup> C Mode (USCI-I2C) ..... | 19        |
| External Bus Interface (EBI) .....                                                     | 20        |
| High Speed USB 2.0 Device Controller (HSUSBD) .....                                    | 20        |
| USB 1.1/2.0 Host Controller (HSUSBH) .....                                             | 21        |
| High Speed USB On-The-Go (HSOTG) .....                                                 | 22        |
| CRC Controller (CRC) .....                                                             | 22        |
| Enhance 12-bit Analog-to-Digital Converter (EADC) .....                                | 22        |
| Analog Comparator Controller (ACMP) .....                                              | 23        |
| Controller Area Network (CAN) .....                                                    | 23        |
| Enhanced Input Capture Timer (ECAP) .....                                              | 24        |
| Enhanced LED Light Strip Interface (ELLSI) .....                                       | 24        |
| LED Light Strip Interface (LLSI) .....                                                 | 24        |
| Secure Digital Host Controller (SDH) .....                                             | 24        |
| Parallel Dimming Communication Interface (PDCI) .....                                  | 25        |
| <b>9 SAMPLECODE\TEMPLATE .....</b>                                                     | <b>26</b> |
| <b>10 SAMPLECODE\TRUSTZONE .....</b>                                                   | <b>27</b> |
| <b>11 SAMPLECODE\XOM.....</b>                                                          | <b>28</b> |

## 1 Document

|                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CMSIS.html</b>                                       | <p>Introduction of CMSIS version 6.1.0 CMSIS components included CMSIS-CORE, CMSIS-Driver, CMSIS-DSP, etc.</p> <ul style="list-style-type: none"><li>● CMSIS-CORE: API for the Cortex-M0 processor core and peripherals.</li><li>● CMSIS-Driver: Defines generic peripheral driver interfaces for middleware making it reusable across supported devices.</li><li>● CMSIS-DSP: DSP Library Collection with over 60 Functions for various data types: fix-point (fractional q7, q15, q31) and single precision floating-point (32-bit).</li></ul> |
| <b>Revision History.pdf</b>                             | The revision history of CM3031 Series BSP.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>NuMicro CM3031 Series Driver Reference Guide.chm</b> | The usage of drivers in CM3031 Series BSP.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## 2 Library

|                      |                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------|
| <b>CMSIS</b>         | Cortex® Microcontroller Software Interface Standard (CMSIS) V6.1.0 definitions by ARM® Corp. |
| <b>CMSIS-DSP</b>     | CMSIS DSP Library.                                                                           |
| <b>Device</b>        | CMSIS compliant device header file.                                                          |
| <b>LlSiYcableLib</b> | LLSI library source code.                                                                    |
| <b>NuMaker</b>       | Specific libraries for CM3031 NuMaker board.                                                 |
| <b>StdDriver</b>     | All peripheral driver header and source files.                                               |
| <b>UsbHostLib</b>    | USB host library source code.                                                                |

### 3 Sample Code

|                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FreeRTOS</b>           | Simple FreeRTOS™ demo code.                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Hard_Fault_Sample</b>  | <p>Show hard fault information when hard fault happened.</p> <p>The hard fault handler show some information included program counter, which is the address where the processor was executing when the hard fault occur. The listing file (or map file) can show what function and instruction that was.</p> <p>It also shows the Link Register (LR), which contains the return address of the last function call. It can show the status where CPU comes from to get to this point.</p> |
| <b>ISP</b>                | Sample codes for In-System-Programming.                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>NuMaker-CM3034HSv2</b> | Sample codes for NuMaker-CM3034HS_V2.0 board.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>PowerManagement</b>    | Power management sample code.                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>StdDriver</b>          | Demonstrate the usage of CM3031 series MCU peripheral driver APIs.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Template</b>           | A project template for CM3031.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>TrustZone</b>          | Includes the demo of secure codes and non-secure codes.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>XOM</b>                | Demonstrate how to create XOM library and use it.                                                                                                                                                                                                                                                                                                                                                                                                                                        |

## 4 ThirdParty

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FatFs</b>    | <p>An open source FAT/exFAT file system library.</p> <p>A generic FAT file system module for small embedded systems. Its official website is: <a href="http://elm-chan.org/fsw/ff/00index_e.html">http://elm-chan.org/fsw/ff/00index_e.html</a>.</p>                                                                                                                                                                                              |
| <b>FreeRTOS</b> | <p>A real time operating system available for free download. Its official website is: <a href="http://www.freertos.org/">http://www.freertos.org/</a>.</p>                                                                                                                                                                                                                                                                                        |
| <b>LibMAD</b>   | <p>A MPEG audio decoder library that currently supports MPEG-1 and the MPEG-2 extension to lower sampling frequencies, as well as the de facto MPEG 2.5 format. All three audio layers — Layer I, Layer II, and Layer III (i.e. MP3) are fully implemented. This library is distributed under GPL license. Please contact Underbit Technologies (<a href="http://www.underbit.com/">http://www.underbit.com/</a>) for the commercial license.</p> |

## 5 SampleCode\ISP

|                   |                                                                                                                    |
|-------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>ISP_CAN</b>    | In-System-Programming Sample code through CAN interface.                                                           |
| <b>ISP_DFU_20</b> | In-System-Programming Sample code through USB interface and following Device Firmware Upgrade Class Specification. |
| <b>ISP_HID_20</b> | In-System-Programming Sample code through USB HID interface.                                                       |
| <b>ISP_I2C</b>    | In-System-Programming Sample code through I <sup>2</sup> C interface.                                              |
| <b>ISP_RS485</b>  | In-System-Programming Sample code through RS485 interface.                                                         |
| <b>ISP_SPI</b>    | In-System-Programming Sample code through SPI interface.                                                           |
| <b>ISP_UART</b>   | In-System-Programming Sample code through UART interface.                                                          |



## 6 SampleCode\NuMaker-CM3034HSv2

|                                 |                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------|
| <b>Blinky</b>                   | Use the timer periodic mode to generate timer interrupt every 1 second.            |
| <b>emWin_GIFDecoder</b>         | To utilize emWin library to demonstrate interactive feature.                       |
| <b>LVGL_WidgetsDemo</b>         | To utilize LVGL v9.4.0 library to demonstrate built-in widgets and material theme. |
| <b>MP3_Player</b>               | MP3 player sample plays MP3 files stored on SD memory card.                        |
| <b>NuvotonAudioGUI_Firmware</b> | NAU88C22 configuration tool main program.                                          |

## 7 SampleCode\PowerManagement

|                                    |                                                                                                                                                    |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SYS_DPDMode_Wakeup</b>          | Show how to wake up system form DPD Power-down mode by Wake-up pin(PC.0), Wake-up Timer, RTC Tick, RTC Alarm or RTC Tamper 0.                      |
| <b>SYS_PowerDown_MinCurrent</b>    | Demonstrate how to minimize power consumption when entering power down mode.                                                                       |
| <b>SYS_PowerDownMode</b>           | Show how to enter to different Power-down mode and wake-up by RTC.                                                                                 |
| <b>SYS_PowerMode</b>               | Show how to set different core voltage and main voltage regulator type.                                                                            |
| <b>SYS_SPDMode_Wakeup</b>          | Show how to wake up system form SPD Power-down mode by GPIO pin(PC.0), Wake-up Timer, Wake-up ACMP, RTC Tick, RTC Alarm, RTC Tamper 0, BOD or LVR. |
| <b>SYS_SPDMode_WakeupAndReturn</b> | Show how to continue executing code after wake-up form SPD Power-down mode by SRAM data retention function.                                        |
| <b>SYS_SPDMode_WakeupVTOR</b>      | Show how to continue executing code after wake-up form SPD Power-down mode by VTOR function.                                                       |

## 8 SampleCode\StdDriver

### System Manager (SYS)

|                           |                                                                                       |
|---------------------------|---------------------------------------------------------------------------------------|
| <b>SYS_BODWakeup</b>      | Show how to wake up system form Power-down mode by brown-out detector interrupt.      |
| <b>SYS_PLLClockOutput</b> | Change system clock to different PLL frequency and output system clock from CLK0 pin. |
| <b>SYS_TrimIRC</b>        | Demonstrate how to use LXT to trim HIRC.                                              |

### Clock Controller (CLK)

|                          |                                                                             |
|--------------------------|-----------------------------------------------------------------------------|
| <b>CLK_ClockDetector</b> | Show the usage of clock fail detector and clock frequency monitor function. |
|--------------------------|-----------------------------------------------------------------------------|

### Flash Memory Controller (FMC)

|                                 |                                                                                                                                                |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FMC_APPROT</b>               | Demonstrate how to use FMC APROM Protect function.                                                                                             |
| <b>FMC_CRC32</b>                | Demonstrate how to use FMC CRC32 ISP Command to calculate the CRC32 checksum of APROM and LDROM.                                               |
| <b>FMC_DualBankFwUpgrade</b>    | Implement a firmware update mechanism based on dual bank flash architecture.                                                                   |
| <b>FMC_ECC</b>                  | Demonstrated FMC ECC status and ECC error fault address.                                                                                       |
| <b>FMC_ExecInSRAM</b>           | Implement a code and execute in SRAM to program embedded Flash (support KEIL MDK only).                                                        |
| <b>FMC_FwUpgradeApplication</b> | Bank remap sample code.                                                                                                                        |
| <b>FMC_IAP</b>                  | Show how to call LDROM function from APROM.                                                                                                    |
| <b>FMC_MultiBoot</b>            | Implement a multi-boot system to boot from different applications in APROM. A LDROM code and 4 APROM code are implemented in this sample code. |
| <b>FMC_ReadAllOne</b>           | Demonstrate how to use FMC Read-All-One ISP command to verify APROM/LDROM pages are all 0xFFFFFFFF or not.                                     |

|                |                                                                                           |
|----------------|-------------------------------------------------------------------------------------------|
| <b>FMC_RW</b>  | Demonstrate how to read/program embedded Flash by ISP function.                           |
| <b>FMC_XOM</b> | This sample code shows how to configure and setup an XOM region the perform XOM function. |

## General Purpose I/O (GPIO)

|                             |                                                                            |
|-----------------------------|----------------------------------------------------------------------------|
| <b>GPIO_EINTAndDebounce</b> | Show the usage of GPIO external interrupt function and de-bounce function. |
| <b>GPIO_EINTTriggerEPWM</b> | Show the usage of GPIO EINT trigger EPWM function.                         |
| <b>GPIO_EINTTriggerPDMA</b> | Show the usage of GPIO EINT trigger PDMA function.                         |
| <b>GPIO_EINTTriggerTPWM</b> | Show the usage of GPIO EINT trigger Timer PWM function.                    |
| <b>GPIO_INT</b>             | Show the usage of GPIO interrupt function.                                 |
| <b>GPIO_OutputInput</b>     | Show how to set GPIO pin mode and use pin data input/output control.       |
| <b>GPIO_PowerDown</b>       | Show how to wake up system from Power-down mode by GPIO interrupt.         |

## PDMA Controller (PDMA)

|                                           |                                                                                    |
|-------------------------------------------|------------------------------------------------------------------------------------|
| <b>PDMA_BasicMode</b>                     | Use PDMA0 Channel 2 to transfer data from memory to memory.                        |
| <b>PDMA_ScatterGather</b>                 | Use PDMA0 channel 4 to transfer data from memory to memory by scatter-gather mode. |
| <b>PDMA_ScatterGather_Ping PongBuffer</b> | Use PDMA0 to implement Ping-Pong buffer by scatter-gather mode (memory to memory). |
| <b>PDMA_StrideRepeat</b>                  | Use PDMA0 channel 0 to transfer data from memory to memory with stride and repeat. |
| <b>PDMA_TimeOut</b>                       | Demonstrate PDMA0 channel 1 get/clear timeout flag with UART1.                     |

## Timer Controller (TIMER)

|                                                  |                                                                                                                                       |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>TIMER_ACMPTrigger</b>                         | Use ACMP to trigger Timer0 counter reset mode.                                                                                        |
| <b>TIMER_CaptureCounter</b>                      | Show how to use the timer2 capture function to capture timer2 counter value.                                                          |
| <b>TIMER_Delay</b>                               | Demonstrate the usage of TIMER_Delay() API to generate a 1 second delay.                                                              |
| <b>TIMER_EventCounter</b>                        | Demonstrates the timer event counter function.                                                                                        |
| <b>TIMER_FreeCountingMode</b>                    | Use the timer0 pin PA.11 to demonstrate timer free counting mode function. And displays the measured input frequency to UART console. |
| <b>TIMER_InterTimerTriggerMode</b>               | Use the timer pin PB.5 to demonstrate inter timer trigger mode function. Also display the measured input frequency to UART console.   |
| <b>TIMER_Periodic</b>                            | Use the timer periodic mode to generate timer interrupt every 1 second.                                                               |
| <b>TIMER_PeriodicINT</b>                         | Implement timer counting in periodic mode.                                                                                            |
| <b>TIMER_PWM_AccumulatorInterruptStopMode</b>    | Demonstrate TIMER PWM accumulator interrupt to stop counting.                                                                         |
| <b>TIMER_PWM_AccumulatorInterruptTriggerPDMA</b> | Demonstrate TIMER PWM accumulator interrupt to trigger PDMA transfer.                                                                 |
| <b>TIMER_PWM_Brake</b>                           | Demonstrate how to use Timer0 PWM brake function.                                                                                     |
| <b>TIMER_PWM_ChangeDuty</b>                      | Change duty cycle and period of output waveform in PWM down count type.                                                               |
| <b>TIMER_PWM_DeadTime</b>                        | Demonstrate Timer PWM Complementary mode and Dead-Time function.                                                                      |
| <b>TIMER_PWM_OutputWaveform</b>                  | Demonstrate output different duty waveform in Timer0~Timer5 PWM.                                                                      |
| <b>TIMER_TimeoutWakeup</b>                       | Use timer to wake up system from Power-down mode periodically                                                                         |
| <b>TIMER_ToggleOut</b>                           | Demonstrate the timer0 toggle out function on pin PB.5.                                                                               |

## Watchdog Timer (WDT)

|                           |                                                                                                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| WDT_TimeoutWakeupAndReset | Implement WDT time-out interrupt event to wake up system and generate time-out reset system event while WDT time-out reset delay period expired. |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|

## Window Watchdog Timer (WWDT)

|                 |                                            |
|-----------------|--------------------------------------------|
| WWDT_CompareINT | Show how to reload the WWDT counter value. |
|-----------------|--------------------------------------------|

## Real Timer Clock (RTC)

|                  |                                                                                  |
|------------------|----------------------------------------------------------------------------------|
| RTC_Alarm_Test   | Demonstrate the RTC alarm function. It sets an alarm 10 seconds after execution. |
| RTC_Alarm_Wakeup | Use RTC alarm interrupt event to wake up system.                                 |
| RTC_Time_Display | Demonstrate the RTC function and displays current time to the UART console.      |

## Basic PWM Generator and Capture Timer (BPWM)

|                     |                                                                                 |
|---------------------|---------------------------------------------------------------------------------|
| BPWM_Capture        | Use BPWM0 Channel 0 to capture the BPWM1 Channel 0 Waveform.                    |
| BPWM_DoubleBuffer   | Change duty cycle and period of output waveform by BPWM Double Buffer function. |
| BPWM_OutputWaveform | Demonstrate how to use BPWM counter output waveform.                            |
| BPWM_SwitchDuty     | Change duty cycle of output waveform by configured period.                      |
| BPWM_SyncStart      | Demonstrate how to use BPWM counter synchronous start function.                 |

## Enhance PWM Generator and Capture Timer (EPWM)

|                                 |                                                      |
|---------------------------------|------------------------------------------------------|
| EPWM_AccumulatorINT_TriggerPDMA | Demonstrate EPWM accumulator interrupt trigger PDMA. |
| EPWM_AccumulatorStopMode        | Demonstrate EPWM accumulator stop mode.              |

|                     |                                                                                                       |
|---------------------|-------------------------------------------------------------------------------------------------------|
| de                  |                                                                                                       |
| EPWM_Brake          | Demonstrate how to use EPWM brake function.                                                           |
| EPWM_Capture        | Capture the EPWM1 Channel 0 waveform by EPWM1 Channel 2.                                              |
| EPWM_DeadTime       | Demonstrate how to use EPWM Dead Zone function.                                                       |
| EPWM_DoubleBuffer   | Change duty cycle and period of output waveform by EPWM Double Buffer function (Period loading mode). |
| EPWM_OutputWaveform | Demonstrate how to use EPWM output waveform.                                                          |
| EPWM_PDMA_Capture   | Capture the EPWM1 Channel 0 waveform by EPWM1 Channel 2, and use PDMA to transfer captured data.      |
| EPWM_SwitchDuty     | Change duty cycle of output waveform by configured period.                                            |
| EPWM_SyncStart      | Demonstrate how to use EPWM counter synchronous start function.                                       |

## Enhanced Quadrature Encoder Interface (EQEI)

|                   |                                          |
|-------------------|------------------------------------------|
| EQEI_CompareMatch | Show the usage of EQEI compare function. |
|-------------------|------------------------------------------|

## UART Interface Controller (UART)

|                   |                                                                    |
|-------------------|--------------------------------------------------------------------|
| UART_AutoBaudRate | Show how to use auto baud rate detection function.                 |
| UART_Autoflow     | Transmit and receive data using auto flow control.                 |
| UART_IrDA         | Transmit and receive data in UART IrDA mode.                       |
| UART_LIN          | Transmit LIN frame including header and response in UART LIN mode. |
| UART_PDMA         | Transmit and receive UART data with PDMA.                          |
| UART_RS485        | Transmit and receive data in UART RS485 mode.                      |
| UART_SingleWire   | Transmit and receive data by UART Single-Wire mode.                |
| UART_TxRxFunction | Transmit and receive data from PC terminal through RS232           |

|                    |                                                                    |
|--------------------|--------------------------------------------------------------------|
|                    | interface.                                                         |
| <b>UART_Wakeup</b> | Show how to wake up system from Power-down mode by UART interrupt. |

## I<sup>2</sup>S Controller (I<sup>2</sup>S)

|                       |                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>I2S_Codec</b>      | This is an I <sup>2</sup> S demo using NAU8822/88L25 audio codec, and used to play back the input from line-in. |
| <b>I2S_Codec_PDMA</b> | This is an I <sup>2</sup> S demo with PDMA function connected with codec.                                       |
| <b>I2S_MP3PLAYER</b>  | MP3 player sample plays MP3 files stored on SD memory card.                                                     |
| <b>I2S_WAVPLAYER</b>  | This is a WAV file player which plays back WAV file stored in SD memory card.                                   |

## I<sup>3</sup>S Controller (I<sup>3</sup>S)

|                       |                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>I3C_Controller</b> | Demonstrates how to initialize the I3C Controller and perform operations to the I3C Target. This sample code needs to use two boards. |
| <b>I3C_Target</b>     | Demonstrates how to initialize the I3C Target and response command from the I3C Controller. This sample code needs to use two boards. |

## Serial Peripheral Interface (SPI)

|                           |                                                                                                                                                                                |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SPI_Flash</b>          | Access SPI flash through SPI interface.                                                                                                                                        |
| <b>SPI_HalfDuplex</b>     | Demonstrate SPI half-duplex mode. SPI0 will be configured as Master mode and SPI1 will be configured as Slave mode. Both SPI0 and SPI1 will be configured as half-duplex mode. |
| <b>SPI_Loopback</b>       | Implement SPI Master loop back transfer. This sample code needs to connect MISO pin and MOSI pin together. It will compare the received data with transmitted data.            |
| <b>SPI_MasterFIFOmode</b> | Configure SPI0 as Master mode and demonstrate how to communicate with an off-chip SPI Slave device with FIFO                                                                   |



|                               |                                                                                                                                                                                                        |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                               | mode. This sample code needs to work with <a href="#">SPI_SlaveFIFOmode</a> sample code.                                                                                                               |
| <b>SPI_PDMA_LoopTest</b>      | Demonstrate SPI data transfer with PDMA. QSPI0 will be configured as Master mode and SPI1 will be configured as Slave mode. Both TX PDMA function and RX PDMA function will be enabled.                |
| <b>SPI_SlaveFIFOmode</b>      | Configure SPI0 as Slave mode and demonstrate how to communicate with an off-chip SPI Master device with FIFO mode. This sample code needs to work with <a href="#">SPI_MasterFIFOmode</a> sample code. |
| <b>SPII2S_Master</b>          | Configure SPI0 as I <sup>2</sup> S Master mode and demonstrate how I <sup>2</sup> S works in Master mode. This sample code needs to work with <a href="#">SPII2S_Slave</a> sample code.                |
| <b>SPII2S_PDMA_Codec</b>      | This is an I <sup>2</sup> S demo with PDMA function connected with audio codec.                                                                                                                        |
| <b>SPII2S_PDMA_Play</b>       | This is an I <sup>2</sup> S demo for playing data and demonstrate how I <sup>2</sup> S works with PDMA.                                                                                                |
| <b>SPII2S_PDMA_PlayRecord</b> | This is an I <sup>2</sup> S demo for playing and recording data with PDMA function.                                                                                                                    |
| <b>SPII2S_PDMA_Record</b>     | This is an I <sup>2</sup> S demo for recording data and demonstrate how I <sup>2</sup> S works with PDMA.                                                                                              |
| <b>SPII2S_Slave</b>           | Configure SPI0 as I <sup>2</sup> S Slave mode and demonstrate how I <sup>2</sup> S works in Slave mode. This sample code needs to work with <a href="#">SPII2S_Master</a> sample code.                 |

## Quad Serial Peripheral Interface (QSPI)

|                            |                                                                                                                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>QSPI_DualMode_Flash</b> | Access SPI flash using QSPI dual mode.                                                                                                                                                                         |
| <b>QSPI_QuadMode_Flash</b> | Access SPI flash using QSPI quad mode.                                                                                                                                                                         |
| <b>QSPI_Slave3Wire</b>     | Configure QSPI0 as Slave 3 wire mode and demonstrate how to communicate with an off-chip SPI Master device with FIFO mode. This sample code needs to work with <a href="#">SPI_MasterFIFOmode</a> sample code. |

## I<sup>2</sup>C Serial Interface Controller (I<sup>2</sup>C)

|                       |                                                                                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| I2C_EEPROM            | Demonstrate how to access EEPROM through a I <sup>2</sup> C interface                                                                                                |
| I2C_Loopback          | Demonstrate how a Master accesses Slave.                                                                                                                             |
| I2C_Master            | Demonstrate how a Master accesses Slave. This sample code needs to work with <a href="#">I2C_Slave</a> sample code.                                                  |
| I2C_MultiBytes_Master | Demonstrate how to use multi-bytes API to access slave. This sample code needs to work with <a href="#">I2C_Slave</a> sample code.                                   |
| I2C_PDMA_TRX          | Demonstrate I2C PDMA mode and need to connect I2C0(Master) and I2C1(Slave).                                                                                          |
| I2C_Slave             | Demonstrate how to set I <sup>2</sup> C in slave mode to receive the data from a Master. This sample code needs to work with <a href="#">I2C_Master</a> sample code. |
| I2C_SMBus             | Demonstrate how to control SMBus interface and use SMBus protocol between Host and Slave.                                                                            |
| I2C_Wakeup_Slave      | Show how to wake up MCU from Power-down mode through I2C interface. This sample code needs to work with <a href="#">I2C_Master</a> sample code.                      |

## Universal Serial Control Interface Controller - UART Mode (USCI-UART)

|                        |                                                                                 |
|------------------------|---------------------------------------------------------------------------------|
| USCI_UART_AutoBaudRate | Show how to use auto baud rate detection function.                              |
| USCI_UART_Autoflow     | Transmit and receive data using auto flow control.                              |
| USCI_UART_PDMA         | Transmit and receive UART data with PDMA.                                       |
| USCI_UART_RS485        | Transmit and receive data in RS485 mode.                                        |
| USCI_UART_TxRxFunction | Transmit and receive data from PC terminal through a RS232 interface.           |
| USCI_UART_Wakeup       | Show how to wake up system from Power-down mode by USCI interrupt in UART mode. |

## Universal Serial Control Interface Controller - SPI Mode (USCI-SPI)

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| USCI_SPI_Loopback | Implement USCI_SPI1 Master loop back transfer. This |
|-------------------|-----------------------------------------------------|

|                            |                                                                                                                                                                                               |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | sample code needs to connect USCI_SPI1_MISO pin and USCI_SPI1_MOSI pin together. It will compare the received data with transmitted data.                                                     |
| <b>USCI_SPI_MasterMode</b> | Configure USCI_SPI1 as Master mode and demonstrate how to communicate with an off-chip SPI Slave device. This sample code needs to work with <a href="#">USCI_SPI_SlaveMode</a> sample code.  |
| <b>USCI_SPI_SlaveMode</b>  | Configure USCI_SPI1 as Slave mode and demonstrate how to communicate with an off-chip SPI Master device. This sample code needs to work with <a href="#">USCI_SPI_MasterMode</a> sample code. |

## Universal Serial Control Interface Controller - I<sup>2</sup>C Mode (USCI-I2C)

|                                   |                                                                                                                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USCI_I2C_EEPROM</b>            | Demonstrate how to access EEPROM through a USCI_I2C interface.                                                                                                                                    |
| <b>USCI_I2C_Master</b>            | Demonstrate how a Master access Slave. This sample code needs to work with <a href="#">USCI_I2C_Slave</a> sample code.                                                                            |
| <b>USCI_I2C_Master_10bit</b>      | Demonstrate how a Master use 10-bit addressing access Slave. This sample code needs to work with <a href="#">USCI_I2C_Slave_10bit</a> sample code.                                                |
| <b>USCI_I2C_Monitor</b>           | Demonstrate how USCI_I2C Monitors transmission between I2C Master and I2C Slave.                                                                                                                  |
| <b>USCI_I2C_MultiBytes_Master</b> | Demonstrate how to use multi-bytes API to access slave. This sample code needs to work with <a href="#">USCI_I2C_Slave</a> sample code.                                                           |
| <b>USCI_I2C_SingleByte_Master</b> | Demonstrate how to use single byte API to access slave. This sample code needs to work with <a href="#">USCI_I2C_Slave</a> sample code.                                                           |
| <b>USCI_I2C_Slave</b>             | Demonstrate how to set I <sup>2</sup> C in slave mode to receive the data from a Master. This sample code needs to work with <a href="#">USCI_I2C_Master</a> sample code.                         |
| <b>USCI_I2C_Slave_10bit</b>       | Demonstrate how to set I <sup>2</sup> C in 10-bit addressing slave mode to receive the data from a Master. This sample code needs to work with <a href="#">USCI_I2C_Master_10bit</a> sample code. |
| <b>USCI_I2C_Wakeup_Slave</b>      | Demonstrate how to set I <sup>2</sup> C to wake up MCU from Power-                                                                                                                                |

down mode. This sample code needs to work [USCI I2C Master](#) sample code.

## External Bus Interface (EBI)

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| <b>EBI_NOR</b>  | Configure EBI interface to access MX29LV320T (NOR Flash) on EBI interface. |
| <b>EBI_SRAM</b> | Configure EBI interface to access BS616LV4017(SRAM) on EBI interface.      |

## High Speed USB 2.0 Device Controller (HSUSBD)

|                                    |                                                                                                                                                                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HSUSBD_Audio10_Codec</b>        | An UAC1.0 sample used to record and play the sound sent from PC through the USB interface.                                                                                                                                                   |
| <b>HSUSBD_Audio10_Headset</b>      | An UAC1.0 sample and used to plays the sound send from PC through the USB interface.                                                                                                                                                         |
| <b>HSUSBD_Audio20_Codec</b>        | An UAC2.0 sample used to record and play the sound sent from PC through the USB interface.                                                                                                                                                   |
| <b>HSUSBD_Audio20_Headset</b>      | An UAC2.0 sample used to play the sound sent from PC through the USB interface.                                                                                                                                                              |
| <b>HSUSBD_HID_Keyboard</b>         | Demonstrate how to implement a USB keyboard device.                                                                                                                                                                                          |
| <b>HSUSBD_HID_Mouse</b>            | Simulate a USB mouse and draws circle on the screen.                                                                                                                                                                                         |
| <b>HSUSBD_HID_Mouse_BC12</b>       | Demonstrate how to implement a USB mouse device with BC1.2 (Battery Charging) which shows different type of charging port after connected USB port. The mouse cursor will move automatically when this mouse device connecting to PC by USB. |
| <b>HSUSBD_HID_MouseKeyboard</b>    | Simulate a USB mouse and a USB keyboard.                                                                                                                                                                                                     |
| <b>HSUSBD_HID_Transfer</b>         | Demonstrate how to transfer data between USB device and PC through USB HID interface. A windows tool is also included in this sample code to connect with a USB device.                                                                      |
| <b>HSUSBD_HID_Transfer_And_MSC</b> | Demonstrate how to implement a composite device (HID Transfer and Mass storage). Transfer data between USB device and PC through the USB HID interface. A windows tool is also included in this sample code to connect with a USB device.    |
| <b>HSUSBD_Mass_Storage_</b>        | USB Mass Storage Device CD-ROM Emulation.                                                                                                                                                                                                    |

|                                          |                                                                                                                                          |
|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CDROM</b>                             |                                                                                                                                          |
| <b>HSUSBD_Mass_Storage_Flash</b>         | Use embedded Data Flash as storage to implement a USB Mass-Storage device.                                                               |
| <b>HSUSBD_Mass_Storage_SactterGather</b> | Demonstrate the usage of USB DMA scatter gather function.                                                                                |
| <b>HSUSBD_Mass_Storage_S D</b>           | Implement a SD card reader.                                                                                                              |
| <b>HSUSBD_Mass_Storage_ShortPacket</b>   | Implement a mass storage class sample to demonstrate how to receive a USB short packet.                                                  |
| <b>HSUSBD_Mass_Storage_S RAM</b>         | Use internal SRAM as back end storage media to simulate a 30 KB USB pen drive.                                                           |
| <b>HSUSBD_VCOM_SerialEmulator</b>        | Demonstrate how to implement a USB virtual com port device.                                                                              |
| <b>HSUSBD_VENDOR_LBK</b>                 | Implement a proprietary Vendor LBK device. This sample requires a CM3031 USB host running sample HSUSBD_USBH_VENDOR_LBK to be connected. |

## USB 1.1/2.0 Host Controller (HSUSBH)

|                                      |                                                                                                                                                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HSUSBH_USBH_AudioClasses</b>      | Demonstrate how to use USBH Audio Class driver. It shows the mute, volume, auto-gain, channel, and sampling rate control.                                                                                      |
| <b>HSUSBH_USBH_DEV_CON N</b>         | Use connect/disconnect callback functions to handle of device connect and disconnect events.                                                                                                                   |
| <b>HSUSBH_USBH_Firmware_Update</b>   | Automatically search and read new firmware from USB drive, if found, update APROM Flash with it.                                                                                                               |
| <b>HSUSBH_USBH_HID</b>               | Use USB Host core driver and HID driver. This sample demonstrates how to submit HID class request and read data from interrupt pipe. This sample supports dynamic device plug/un-plug and multiple HID devices |
| <b>HSUSBH_USBH_HID_Keyboard</b>      | Demonstrate reading key inputs from USB keyboards. This sample includes an USB keyboard driver which is based on the HID driver.                                                                               |
| <b>HSUSBH_USBH_HIDMouse_Keyboard</b> | Demonstrates how to support USB mouse and keyboard input.                                                                                                                                                      |
| <b>HSUSBH_USBH_MassStorage</b>       | Use a command-shell-like interface to demonstrate how to use USBH mass storage driver and make it work as a disk driver                                                                                        |

|                          |                                                                                                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ge                       | under the FATFS file system.                                                                                                                                               |
| HSUSBH_USBH_UAC_HID      | Show how to use USBH Audio Class driver and HID driver at the same time. The target device is a Game Audio (UAC+HID composite device).                                     |
| HSUSBH_USBH_UAC_LoopBack | Receive audio data from an UAC device, and immediately send back to the UAC device.                                                                                        |
| HSUSBH_USBH_VCOM         | Demonstrate how to use the USB Host core driver and CDC driver to connect a CDC class VCOM device.                                                                         |
| HSUSBH_USBH_VENDOR_LBK   | Show how to do transfer on a known device with a vendor driver. This sample requires a CM3031 USB device running sample <a href="#">HSUSBD_VENDOR_LBK</a> to be connected. |

## High Speed USB On-The-Go (HSOTG)

|                      |                                                                                                                                                                                                                               |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HSOTG_Dual_Role_UMAS | An OTG sample code that will become a USB host when connected with a Micro-A cable, and can access the pen drive when plugged in. It will become a removable disk when connected with a Micro-B cable, and then plug into PC. |
| HSOTG_HNP            | Show HID mouse with OTG HNP protocol.                                                                                                                                                                                         |

## CRC Controller (CRC)

|           |                                                                  |
|-----------|------------------------------------------------------------------|
| CRC_CCITT | Implement CRC in CRC-CCITT mode and get the CRC checksum result. |
| CRC_CRC8  | Implement CRC in CRC-8 mode and get the CRC checksum result.     |
| CRC_CRC32 | Implement CRC in CRC-32 mode with PDMA transfer.                 |

## Enhance 12-bit Analog-to-Digital Converter (EADC)

|                    |                                                                  |
|--------------------|------------------------------------------------------------------|
| EADC_Accumulate    | Demonstrate how to get accumulate conversion result.             |
| EADC_ADINT_Trigger | Use ADINT interrupt to do the EADC continuous scan conversion.   |
| EADC_Average       | Demonstrate how to get average conversion result.                |
| EADC_BandGap       | Convert Band-gap (Sample module 16) and print conversion result. |

|                               |                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>EADC_PDMA_EPWM_Trigger</b> | Demonstrate how to trigger EADC by EPWM and transfer conversion data by PDMA.                       |
| <b>EADC_EPWM_Trigger</b>      | Demonstrate how to trigger EADC by EPWM.                                                            |
| <b>EADC_Pending_Priority</b>  | Demonstrate how to trigger multiple sample modules and got conversion results in order of priority. |
| <b>EADC_ResultMonitor</b>     | Monitor the conversion result of channel 2 by the digital compare function.                         |
| <b>EADC_SWTRG_Trigger</b>     | Trigger EADC by writing EADC_SWTRG register.                                                        |
| <b>EADC_TempSensor</b>        | Convert temperature sensor (Sample module 17) and print conversion result.                          |
| <b>EADC_Timer_Trigger</b>     | Show how to trigger EADC by timer.                                                                  |

## Analog Comparator Controller (ACMP)

|                           |                                                                        |
|---------------------------|------------------------------------------------------------------------|
| <b>ACMP_CompareVBG</b>    | Demonstrate how ACMP compare VBG output with ACMP1_P1 value.           |
| <b>ACMP_Wakeup</b>        | Show how to wake up MCU from Power-down mode by ACMP wake-up function. |
| <b>ACMP_WindowCompare</b> | Demonstrate the usage of ACMP window compare function.                 |

## Controller Area Network (CAN)

|                                |                                                                  |
|--------------------------------|------------------------------------------------------------------|
| <b>CANFD_CAN_Loopback</b>      | Use CAN mode function to do internal loopback test.              |
| <b>CANFD_CAN_MonitorMode</b>   | Use CAN Monitor mode to listen to CAN bus communication test.    |
| <b>CANFD_CAN_TxRx</b>          | Transmit and receive CAN message through CAN interface.          |
| <b>CANFD_CAN_TxRxINT</b>       | An example of interrupt control using CAN bus communication.     |
| <b>CANFD_CANFD_Loopback</b>    | Use CAN FD mode function to do internal loopback test.           |
| <b>CANFD_CANFD_MonitorMode</b> | Use CAN FD Monitor mode to listen to CAN bus communication test. |
| <b>CANFD_CANFD_TxRx</b>        | Transmit and receive CAN FD message through CAN interface.       |



|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| CANFD_CANFD_TxRxINT | An example of interrupt control using CAN FD bus communication. |
|---------------------|-----------------------------------------------------------------|

## Enhanced Input Capture Timer (ECAP)

|                   |                                                       |
|-------------------|-------------------------------------------------------|
| ECAP_GetInputFreq | Show how to use ECAP interface to get input frequency |
| ECAP_GetQEIFreq   | Show how to use ECAP interface to get QEI frequency.  |

## Enhanced LED Light Strip Interface (ELLSI)

|                            |                                                                                                                                                                                    |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ELLSI_Marquee              | This is a ELLSI demo for marquee display in software mode. It needs to be used with WS2812 LED strip.                                                                              |
| ELLSI_Marquee_Y_Cable      | This is a ELLSI demo for marquee display in software mode. It needs to be used with AP6112Y LED strip. The AP6112Y LED strip will be initialized through manual software settings. |
| ELLSI_PDMA_Marquee         | This is a ELLSI demo for marquee display in PDMA mode. It needs to be used with WS2812 LED strip.                                                                                  |
| ELLSI_PDMA_Marquee_Y_Cable | This is a ELLSI demo for marquee display in PDMA mode. It needs to be used with AP6112Y LED strip. The AP6112Y LED strip will be initialized through automatic hardware settings.  |

## LED Light Strip Interface (LLSI)

|                      |                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------|
| LLSI_Marquee         | This is a LLSI demo for marquee display in software mode. It needs to be used with WS2812 LED strip. |
| LLSI_PDMA_Marquee    | This is a LLSI demo for marquee display in PDMA mode. It needs to be used with WS2812 LED strip.     |
| LLSI_Y_Cable_Control | This is a LLSI demo for Y-cable control. It needs to be used with AP6112Y LED strip.                 |

## Secure Digital Host Controller (SDH)



|                                                        |                                                 |
|--------------------------------------------------------|-------------------------------------------------|
| SDH_FATFS                                              | Access a SD card formatted in FAT file system.  |
| <b>Parallel Dimming Communication Interface (PDCI)</b> |                                                 |
| PDCI_Output                                            | Demonstrate PDCI Mode0 data transfer.           |
| PDCI_PDMA_Output                                       | Demonstrate PDCI Mode0 data transfer with PDMA. |

## 9 SampleCode\Template

| Template |
|----------|
|----------|

|                                    |
|------------------------------------|
| A project template for CM3031 MCU. |
|------------------------------------|

## 10 SampleCode\TrustZone

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| <b>HardFault</b> | Show the hard fault usages in both secure and non-secure code.   |
| <b>Template</b>  | Demonstrate the how to implement code for secure and non-secure. |

## 11 SampleCode\XOM

|            |                                                 |
|------------|-------------------------------------------------|
| XOMLib     | Demonstrate how to create XOM library.          |
| XOMLibDemo | Demonstrate how to use <a href="#">XOMLib</a> . |

### **Important Notice**

**Nuvoton Products are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".**

**Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.**

**All Insecure Usage shall be made at customer's risk, and in the event that third parties lay claims to Nuvoton as a result of customer's Insecure Usage, customer shall indemnify the damages and liabilities thus incurred by Nuvoton.**

---

*Please note that all data and specifications are subject to change without notice.  
All the trademarks of products and companies mentioned in this datasheet belong to their respective owners.*